Nivel 1

Ejercicio 1

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

```sql
use transactions;

drop table if exists credit_card;
create table credit_card (
    id varchar(20) primary key,
    iban varchar(40),
    pan varchar(20),
    pin varchar(5),
    cvv varchar(5),
    expiring_date varchar(10)
);
```

Output

Action Output

#	Time	Action	Message
1	16:03:47	use transactions	0 row(s) affected
2	16:03:47	drop table if exists credit_card	0 row(s) affected
3	16:03:47	create table credit_card (id varchar(20) primary key, iban varchar(40), pan varchar(20), pin varchar(5), cvv varchar(5), expiring_date varch...	0 row(s) affected

Corremos información de archivo "datos_introducir_credit"

```sql
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS1193982162957159683424568 21', '3784 66
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 137
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143310514852179535', '455676 64374
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '402400712
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3057', 'LU931822574697545215', '3484 621767 2123
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3064', 'PS1469655454492533776272731 33', '3467 73
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3071', 'NO8923814763512', '3464 789562 23352', '
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3078', 'IS0251271458846232795487 33', '4539 322 7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3085', 'BE63114723972437', '5266 3346 1135 1687'
```

Output

Action Output

#	Time	Action	Message
4989	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9570', 'XX6339420102582861815165 73', '7233675965634598', '5619'...	1 row(s) affected
4990	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9571', 'XX4849153824371419965794 68', '1265066247957405', '6585'...	1 row(s) affected
4991	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9572', 'XX4054481862697303884045 08', '8296622848225053', '1995'...	1 row(s) affected
4992	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9573', 'XX9653389310530889019249 06', '9099040179637171', '2277'...	1 row(s) affected
4993	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9574', 'XX6627617583614326865207 75', '3171178331318656', '5084'...	1 row(s) affected
4994	16:00:59	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9575', 'XX4998812160736265711964 73', '8086033007377786', '7256'...	1 row(s) affected

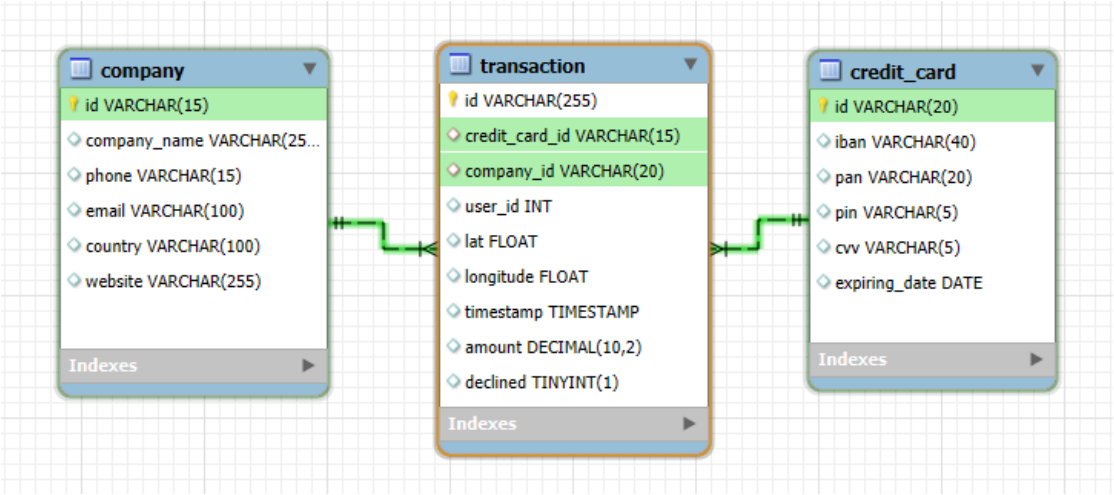
Cambiamos el formato de expiring_date de str a date y relacionamos tablas credit_card y transaction.

```sql
update credit_card
set expiring_date = str_to_date(expiring_date, '%m/%d/%y')
where id != '';

alter table credit_card
modify expiring_date date; -- para que se modifique en la tabla

alter table transaction
add constraint card_transaction
foreign key (credit_card_id)
references credit_card(id);
```

Output

Action Output

#	Time	Action
1	16:05:14	update credit_card set expiring_date = str_to_date(expiring_date, '%m/%d/%y') where id != ''
2	16:05:14	alter table transaction add constraint card_transaction foreign key (credit_card_id) references credit_card(id)
3	16:10:08	SELECT * FROM transactions.credit_card



El diagrama muestra un modelo relacional sencillo de tipo estrella para registrar transacciones con tarjetas de crédito: la tabla company, que actúa como tabla dimensional, almacena información básica de las empresas (nombre, contacto y país); la tabla credit_card, también dimensional, guarda los datos de las tarjetas (IBAN, PAN, PIN, CVV y fecha de expiración); y la tabla transaction actúa como tabla de hechos, registrando cada transacción con su identificador, la tarjeta utilizada (credit_card_id), la empresa asociada (company_id), el usuario, la ubicación (latitud y longitud), la fecha/hora, el monto y si fue rechazada. Las relaciones indican que una empresa y una tarjeta pueden estar asociadas a muchas transacciones, mientras que cada transacción corresponde a una única empresa y una única tarjeta.

Ejercicio 2

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

```
update credit_card
set iban = 'TR323456312213576817699999'
where id = 'CcU-2938';

select iban
from credit_card
where id = 'CcU-2938';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban
TR323456312213576817699999

credit_card 1

Output

Action Output

#	Time	Action
1	16:33:16	update credit_card set iban = 'TR323456312213576817699999' where id = 'CcU-2938'
2	16:33:16	select iban from credit_card where id = 'CcU-2938'

Ejercicio 3

En la tabla "transaction" ingresa una nueva transacción con la siguiente información:

Id	108B1D1D-5B23-A76C-55EF-C568E49A99DD
credit_card_id	CcU-9999
company_id	b-9999
user_id	9999
latitud	829.999
longitud	-117.999
amount	111.11
declined	0

```
insert into credit_card (id)  values ('CcU-9999');
insert into company (id)  values ('b-9999');
insert into transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
values ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
```

Output

Action Output

#	Time	Action	Message
1	16:43:12	insert into credit_card (id) values ('CcU-9999')	1 row(s) affected
2	16:43:12	insert into company (id) values ('b-9999')	1 row(s) affected
3	16:43:12	insert into transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) values ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', '...	1 row(s) affected

Ejercicio 4

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

```
alter table credit_card
drop column pan;

select *
from credit_card;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
CcS-4859	XX7826930491423553609370	4983	277	2026-11-26
CcS-4860	XX5559590368835304645299	6876	661	2027-07-27
CcS-4861	XX2035182877195191627307	5710	398	2026-04-25
CcS-4862	XX4774721462463645409758	4042	174	2026-11-27

credit_card 2

Output

Action Output

#	Time	Action
1	16:48:38	alter table credit_card drop column pan
2	16:48:38	select * from credit_card

Nivel 2

Ejercicio 1

Eliminar el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la tabla de transacciones.

```
delete from transaction
where id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Output

Action Output

#	Time	Action
1	16:51:51	delete from transaction where id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'

Ejercicio 2

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

```
create view vistamarketing as
select c.company_name,
       c.phone,
       c.country,
       round(avg(t.amount),2) as average_purchase
from company c
join transaction t on c.id = t.company_id
group by c.id, c.company_name
order by average_purchase desc;

select *
from vistamarketing;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	average_purchase
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Pretium Neque Corp.	07 77 48 55 28	Australia	276.16
Urna Convallis Associates	06 01 24 77 04	United States	274.24
At Associates	09 56 61 10 65	New Zealand	272.21
Metus Vitae Associates	08 25 44 40 66	Australia	270.08

stamarketing 1

Output

Action Output

#	Time	Action	Message
1	18:05:03	create view vistamarketing as select c.company_name, c.phone, c.country, round(avg(t.amount),2) as average_purchase from company...	0 row(s) affected
2	18:05:03	select * from vistamarketing	101 row(s) returned

Ejercicio 3

Filtra la vista VistaMarketing para mostrar sólo las compañías que tienen su país de residencia en "Germany"

```
select *
from vistamarketing
where country = "Germany";
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	average_purchase
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Nunc Interdum Incorporated	05 18 15 48 13	Germany	259.32
Convallis In Incorporated	06 66 57 29 50	Germany	257.75
Ac Industries	09 34 65 40 60	Germany	255.15
Rutrum Non Inc.	02 66 31 61 09	Germany	255.14

vistamarketing 3

Output

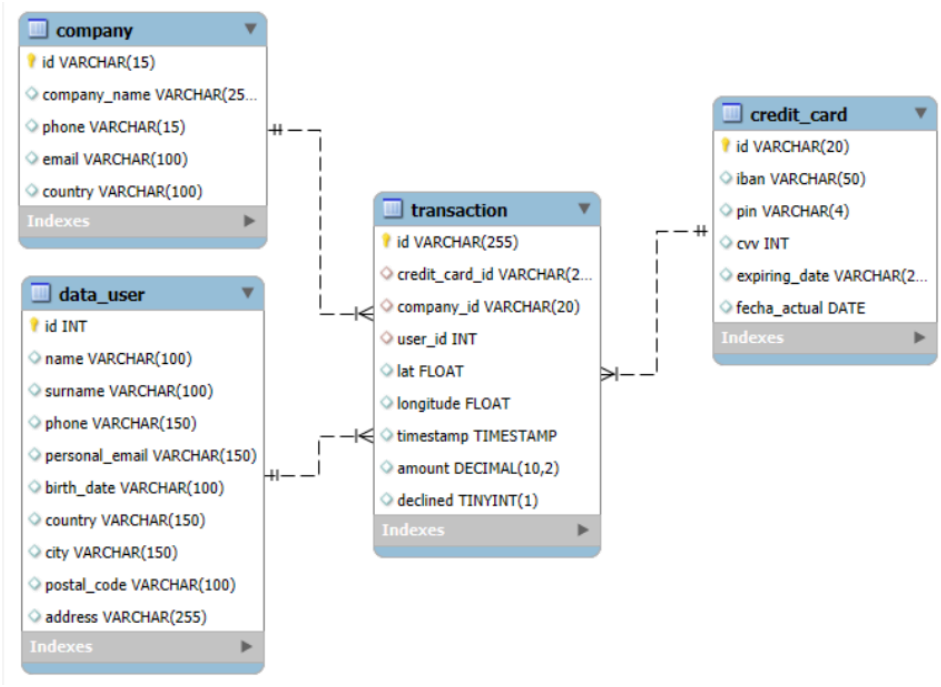
Action Output

#	Time	Action	Message
1	18:05:59	select * from vistamarketing where country = "Germany"	8 row(s) returned

Nivel 3

Ejercicio 1

La próxima semana tendrás una nueva reunión con los gerentes de marketing. Un compañero de tu equipo realizó modificaciones en la base de datos, pero no recuerda cómo las realizó. Te pide que le ayudes a dejar los comandos ejecutados para obtener el siguiente diagrama:

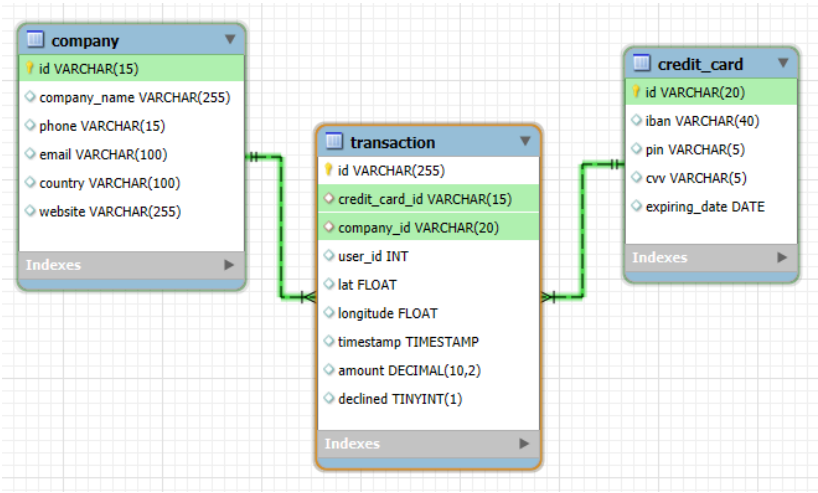


Recordatorio

En esta actividad, es necesario que describas el "paso a paso" de las tareas realizadas. Es importante realizar descripciones sencillas, simples y fáciles de comprender. Para realizar esta actividad deberás trabajar con los archivos denominados "estructura_datos_user" y "datos_introducir_user".

Recuerda seguir trabajando sobre el modelo y las tablas con las que ya has trabajado hasta ahora.

Modelo a transformar:



1) Corremos "estructura_datos_user" para crear tabla **user** y ejecutamos "datos_introducir_user" para insertar datos.

```sql
CREATE TABLE IF NOT EXISTS user (
    id CHAR(10) PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(150),
    email VARCHAR(150),
    birth_date VARCHAR(100),
    country VARCHAR(150),
    city VARCHAR(150),
    postal_code VARCHAR(100),
    address VARCHAR(255)
);
```

Output

Action Output

#	Time	Action	Message
1	14:07:35	CREATE TABLE IF NOT EXISTS user (id CHAR(10) PRIMARY KEY, ...	0 row(s) affected

```sql
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
INSERT INTO user (id, name, surname, phone, email, birth_date, country, city,
```

Output

Action Output

#	Time	Action	Message
4983	14:08:44	INSERT INTO user (id, name, surname, phone, email, birth_date, cou...	1 row(s) affected
4984	14:08:44	INSERT INTO user (id, name, surname, phone, email, birth_date, cou...	1 row(s) affected
4985	14:08:44	INSERT INTO user (id, name, surname, phone, email, birth_date, cou...	1 row(s) affected
4986	14:08:44	INSERT INTO user (id, name, surname, phone, email, birth_date, cou...	1 row(s) affected

2) En tabla **user** hacemos:
   - cambio nombre de user a data_user
   - pasar id char(10) a int
   - cambio nombre email a personal_email

```sql
rename table user to data_user;
alter table data_user
modify column id int,
rename column email to personal_email;
```

Output

Action Output

#	Time	Action
1	14:22:44	rename table user to data_user
2	14:22:44	alter table data_user modify column id int, rename column email to personal_email

3) En tabla **company** eliminamos columna website.

```sql
alter table company
drop column website;
```

Output

Action Output

#	Time	Action
1	14:33:47	alter table company drop column website

4) En tabla **transaction** cambiamos credit_card_id varchar(15) a varchar(20).

```sql
alter table transaction
modify column credit_card_id varchar(20);
```

Output

Action Output

#	Time	Action
1	14:34:54	alter table transaction modify column credit_card_id varchar(20)

5) En tabla **credit_card** cambiar:
   - iban varchar(40) a varchar(50)
   - pin varchar(5) a varchar(4)
   - cvv varchar(5) a int
   - expiring_date date a varchar(20)
   - agregar columna fecha_actual date

```
145 •  alter table credit_card
146    modify column iban varchar(50),
147    modify column pin varchar(4),
148    modify column cvv int,
149    modify column expiring_date varchar(20),
150    add column fecha_actual date;
151
152
```

Output

Action Output

#	Time	Action
1	14:36:10	alter table credit_card modify column iban varchar(50), modify column pin varchar(4), modify column cvv int, modify column expiring_date varchar(20), add column fecha_actual date

6) Buscamos datos huérfanos en columna id de tabla **data_user** y lo agregamos a la tabla.

```
154 •  select distinct t.user_id
155    from transaction t
156    left join data_user d on t.user_id = d.id
157    where d.id is null;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

user_id
9999

Result 3

Output

Action Output

#	Time	Action
1	14:57:30	select distinct t.user_id from transaction t left join data_user d on

```
159 •  insert into data_user (id) values (9999);
160
```

Output

Action Output

#	Time	Action
1	14:54:30	insert into data_user (id) values (9999)

7) Creamos relación entre **data_user** y **transaction**.

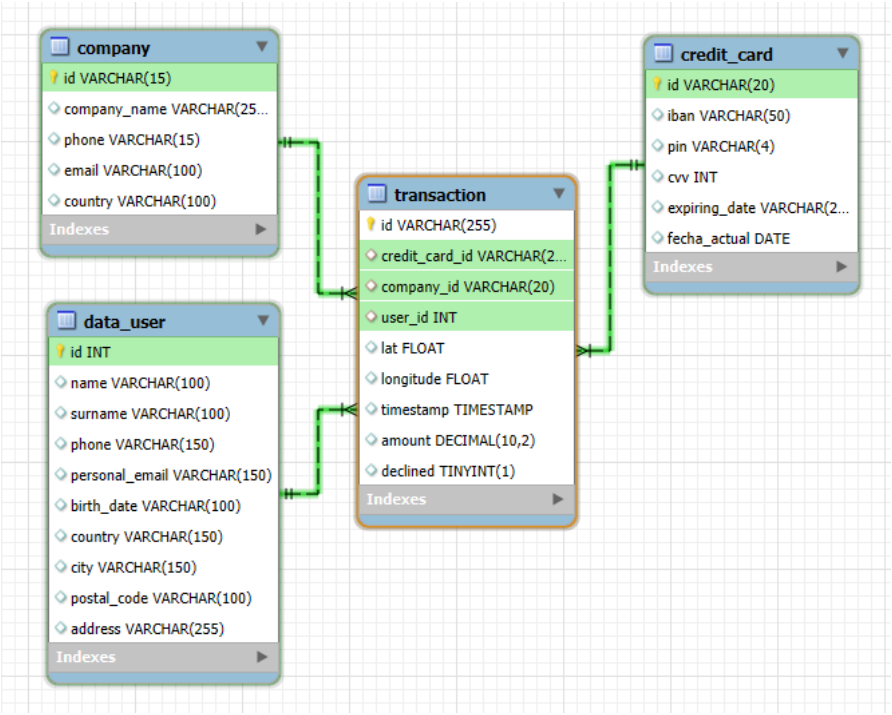
```
163 •  alter table transaction
164    add constraint user_transaction
165    foreign key (user_id)
166    references data_user(id);
167
```

Output

Action Output

#	Time	Action
1	15:02:12	alter table transaction add constraint user_transaction foreign key (user_id) references data_user(id)

Modelo transformado:



Ejercicio 2

La empresa también le pide crear una vista llamada "InformeTecnico" que contenga la siguiente información:

- ID de la transacción
- Nombre del usuario/a
- Apellido del usuario/a
- IBAN de la tarjeta de crédito usada.
- Nombre de la compañía de la transacción realizada.
- Asegúrese de incluir información relevante de las tablas que conocerá y utilice alias para cambiar de nombre columnas según sea necesario.

```
create view informetecnico as
select t.id as "id de transaccion",
       u.name,
       u.surname,
       cc.iban,
       c.company_name
from transaction t
join company c on c.id = t.company_id
join data_user u ON t.user_id = u.id
join credit_card cc ON cc.id = t.credit_card_id;
```

Output

Action Output

#	Time	Action	Message
1	18:11:37	create view informetecnico as select t.id as "id de transaccion", u.name, u.surname, cc.iban, c.company_name from transaction t join co...	0 row(s) affected

Muestra los resultados de la vista, ordena los resultados de forma descendente en función de la variable ID de transacción.

```
select *
from informetecnico;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id de transaccion	name	surname	iban	company_name
00D8F1C6-FF88-4E47-B26B-BC32AD4112C3	Zeus	Gamble	TR3738725583135456671242 86	A Institute
02800DB7-FAF0-4602-B28C-76FF7F8897EE	Zeus	Gamble	TR3738725583135456671242 86	Malesuada PC
1EDDC7AC-D3CA-440B-A048-3CBC692995CB	Zeus	Gamble	TR3738725583135456671242 86	Pede Cum Ltd
338C8AEC-D406-4D20-9B83-00503AACA5AB	Zeus	Gamble	TR3738725583135456671242 86	Risus Associates
3B5E6817-317E-4C92-A7C8-B50F7571858D	Zeus	Gamble	TR3738725583135456671242 86	Auctor Mauris Vel LLP
3D2869C9-52D8-4903-9666-08CE556A5054	Zeus	Gamble	TR3738725583135456671242 86	Neque Tellus Incorporated
3F2720A9-C1C9-4D9B-9FFD-C822367D6929	Zeus	Gamble	TR3738725583135456671242 86	Neque Tellus Imperdiet Corp.
4AB68FF1-181A-4690-AB53-92B865C7E89C	Zeus	Gamble	TR3738725583135456671242 86	Magna Incorporated

informetecnico 5

Output

Action Output

#	Time	Action	Message
1	18:12:07	select * from informetecnico	100000 row(s) returned